

实战：小红书自动化发布 系统 UI 穿透与浏览器自动化

当业务系统没有 API，Agent 如何进入用户界面（UI）完成动作

彭靖田 | 谷歌 AI 开发者专家 · 上海载极数据科技 CEO

本节 课程地图

先看效果，再拆能力栏，再走代码，最后跑通 draft 链路

1

没有 API? 界面就是接口

先看 Agent 如何越过封闭后台的最后一公里

2

执行层的能力从哪来

从 CUA 到 CDP，理解浏览器自动化的技术底座

3

锁定业务对象，而不是追逐像素

语义锚点、多候选 Selector 与抗变动定位策略

4

一键部署、对话发布、扫码接管

三步跑通小红书自动发布：部署 Skill → 飞书对话 → 扫码登录

5

实战：在飞书跑通小红书全自动发布

手把手带你完成实战闭环

6

总结与思考

四条核心 Takeaway 与一次完整自动化上传练习

第 1 章

没有 API? 界面就是接口

先看效果: Agent 进入小红书封闭后台, 完成填写, 停在发布前

目标：Agents 自动化完成小红书发布

- **输入：**一份图文内容 JSON
- **过程：**浏览器打开封闭创作后台，检查登录态，必要时生成扫码二维码
- **输出：**图片上传、标题填写、正文填写、话题追加。

① 可能含AI生成内容

OpenClaw 商业应用场景

没有 API 的后台，也能自动跑流程

OpenClaw 能帮业务团队做什么？

很多团队不是缺 AI 聊天工具，而是缺一个能把业务流程真正跑起来的数字员工。OpenClaw 的价值在于：它可以长期驻留、记住上下文、按计划执行任务，并通过浏览器自动化接入那些没有 API 的后台系统。

典型场景包括：

1. 运营团队：自动收集竞品动态、整理素材、生成发布草稿。
2. 销售团队：从会议纪要和客户资料里提取线索，自动更新 CRM。
3. 财务团队：读取票据和表格，再登录后台完成填报和截图留档。
4. 管理团队：每天早上自动生成业务简报，把异常项推送给负责人。

它最适合解决的不是单次问答，而是重复、跨系统、有检查闭环的业务流。简单说：把人从机械操

评论 点赞 收藏 评论 活转

① 可能含AI生成内容

OpenClaw 在工业现场更有用

不仅是 AI 聊天，更是设备连接与流程执行

OpenClaw 工业场景应用

这段时间看了不少 AI 落地场景，我越来越确定一件事：

工业场景真正缺的，不是更会聊天的 AI。而是一个能接消息、接工具、接流程、接执行动作的系统。

很多企业并不缺系统，也不缺数据。真正缺的，是出了问题之后，谁来把这些东西快速串起来。

设备异常、巡检记录、工单流转、群里沟通、SOP 文档、负责人确认..... 这些信息其实都在，但散在各处。最后最消耗人的，往往不是分析本身，而是：

- 找信息
- 追进度

评论 点赞 收藏 评论 活转

Agents 自动化小红书发布流程

授权账号 → 人工扫码 → UI 穿透执行 → 发布前可停住

输入：图文内容 JSON



```
{
  "title": "秋日穿搭分享 | 温柔针织+半裙的 5 种搭配",
  "content": "🍂 秋天来了，分享 5 套温柔显气质的穿搭...",
  "images": [
    "https://xxx.com/img1.jpg",
    "https://xxx.com/img2.jpg"
  ],
  "topics": ["#秋日穿搭", "#温柔风", "#小个子穿搭"],
  "publishTime": "now"
}
```

浏览器打开封闭创作后台

1 打开后台

打开小红书创作服务平台
(封闭后台)



2 检查登录态

检查 Cookie / 登录状态
是否有效



3 需要人工授权

展示二维码
人工扫码登录



4 登录成功

获取 Cookie
建立会话



执行发布流程 (UI 穿透)

5 图片上传

上传 images[] 中的图片



✓ 已完成

6 填写标题

填写 title



✓ 已完成

7 填写正文

填写 content



✓ 已完成

8 追加话题

追加 topics[]



✓ 已完成

9 发布前预览 (可停住)

展示预览, 人工确认后发布
可修改 / 可取消 / 可暂停



等待人工确认



10 发布

点击发布按钮



待发布

UI 穿透执行链路 (可视化 + 可控)

1 打开后台



2 检查登录态



3 人工扫码登录



4 登录成功



5 图片上传



6 填写标题



7 填写正文



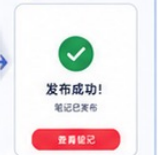
8 追加话题



9 发布前预览



10 发布



授权账号 (长期有效)

- 首次使用需要人工扫码授权
- Cookie 安全加密存储
- 支持多账号管理



人工介入 (安全可控)

- 仅登录时需要人工扫码
- 发布前可预览、可修改、可暂停
- 确认无误后再发布



自动化执行 (可靠稳定)

- UI 穿透执行, 模拟真实操作
- 断点续传, 失败可重试
- 详细日志, 问题可追溯



状态通知 (实时同步)

- 登录状态变化通知
- 发布结果实时通知
- 异常情况告警通知

把内容封装成可执行任务对象

/examples/openclaw_business_content.json

```
{  
  "title": "OpenClaw 能帮业务团队做什么? ",  
  "body": "很多团队不是缺 AI 聊天工具,  
          而是缺一个能把业务流程真正跑起来的数字员工。...",  
  "topics": ["OpenClaw", "AI工具", "浏览器自动化"],  
  "images": ["assets/Openclaw商业应用场景.jpg"],  
  "mode": "publish",  
}
```

业务内容

title / body / topics 是要交付给后台的内容，不是提示词。

真实文件

images 指向需要上传的本地素材，执行器必须能访问。

执行模式

mode=publish 正式提交。

效果预览：Agent 进入小红书后台操作

标题来自 JSON 的 title

正文来自 JSON 的 body

话题自动追加为 #OpenClaw #AI工具

图片通过上传控件写入页面



小红书 UI 穿透执行架构：从任务到后台动作

自动化执行 + 人工可控接管 + 结果可回溯闭环



如果模型已经很强，为什么还要建设 UI 执行层？

01 它们能不能打开你的创作后台？

系统入口

02 它们能不能上传本地图片并处理登录态？

真实动作

03 它们能不能在云服务器上等待你扫码后继续执行？

接管闭环

模型智能解决“想清楚”，UI 执行层解决“做完它”。

模型层 vs 执行层：两种完全不同的能力

模型层

内容生成

生成标题、正文、摘要、改写。

规则解释

解释流程规则，给出操作建议。

边界限制

不直接接管真实业务系统。

UI 执行层

进入系统

打开真实后台页面。

完成动作

上传文件、填写表单、点击按钮。

可接管闭环

保留登录态，等待扫码，记录结果。

AI 能力分层：从内容智能 Chatbot 到系统执行 Agent

能力	聊天 AI (Kimi/豆包)	Agent + 浏览器自动化
生成运营文案	强	可调用模型生成
生成标题候选	强	可作为前置步骤
打开封闭后台	通常不能	可以
上传本地图片	通常不能	可以
保留登录态	不适用	可用浏览器 profile
二维码人工接管	不适用	可生成截图与 payload
执行结果留痕	有限	可保存截图、DOM、JSON

模型层解决生成与判断，UI 穿透层解决系统内执行。

第 2 章

执行层的能力从哪来

从 CUA 到 CDP，理解浏览器自动化的技术底座

“

AI 业务流的质变，不发生在回答更聪明的时候，而发生在它能进入系统完成动作的时候。

—— AI 业务流架构师课程方法论

模型负责想清楚，执行层负责做完它——思考层、工具层、执行层，必须架构到同一条链路里。

CUA: Computer-Using Agent



CUA 可以理解为让 Agent 使用计算机界面完成任务。它包含三个能力环：

观察

截图、DOM、页面状态、控件信息

决策

识别下一步要做什么

执行

点击、输入、上传、切换页面

只会生成文字，Agent 永远停在建议层；能操作界面，才进入执行层。

系统 UI 穿透的四层能力栈

API 自动化

有官方接口就直接调用，稳定、可监控、速度快。

浏览器 DOM 穿透

没有 API，但网页结构可读，用 Playwright/CDP 操作页面元素。本节课的主要路径。

视觉锚点控制

DOM 不稳定或不可读时，通过截图、OCR、视觉锚点判断位置。

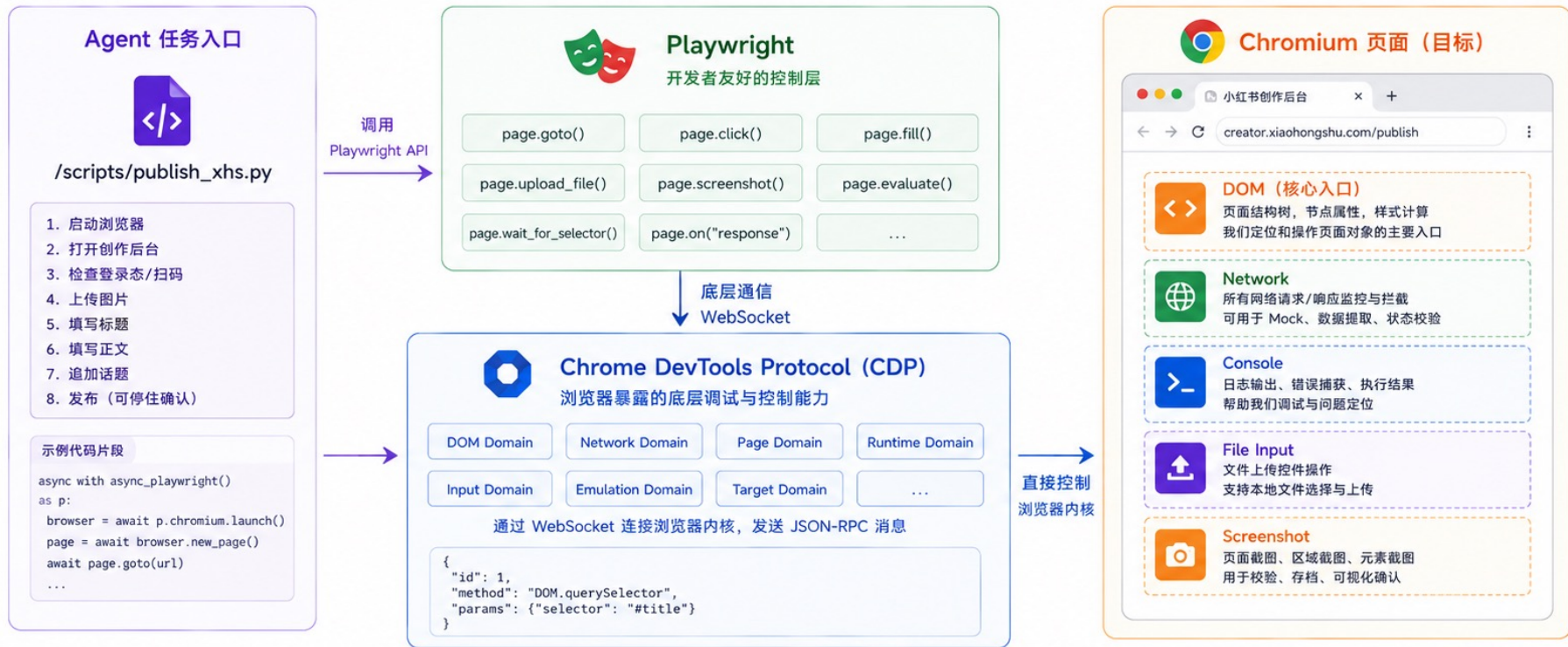
原生 UI 桥接

浏览器之外的桌面软件，需要系统权限和本地伴侣应用（如 Peekaboo Bridge）。本节不涉及。

优先级从上到下——能用结构化能力就不要退回纯视觉。

CDP 浏览器内核级控制通道

从 Agent 脚本到浏览器内核的完整控制链路



关键说明



Playwright 是开发者友好的控制层

封装了复杂的 CDP 调用，提供简洁易用的 API
让自动化脚本更稳定、可维护



CDP 是浏览器暴露的底层能力

提供了对浏览器内核的全量控制能力
包括 DOM、Network、输入、渲染等



DOM 是我们定位和操作页面对象的主要入口

通过选择器定位元素，读取/修改属性，触发事件
是自动化操作的核心基础

DOM 操作和视觉操作，各有适用场景

DOM 操作

优点

稳定、可验证、能读控件属性

适合

网页表单、上传控件、按钮、文本节点

风险

前端重构后 selector 可能失效

视觉操作

优点

接近人类观察方式，不依赖完整 DOM

适合

复杂画布、远程桌面、原生软件

风险

分辨率、遮挡、主题色、缩放都影响

本节验证项目主要走 DOM + 截图留痕，不是纯视觉点击。

Web UI 与原生 UI: 两条不同的接管路径

同一 Agent, 不同通道, 适用于不同的场景与能力边界

路径一: Web UI 路径 (本节验证场景)

✓ 本节验证场景走 Web UI 路径



路径二: 原生 UI 路径 (Peekaboo 扩展边界)

📦 Peekaboo 是后续扩展边界



关键对比



	控制对象	技术栈	权限要求	稳定性	适用场景	本节状态
Web UI 路径	浏览器 DOM	Playwright + CDP	无需系统级权限	高 (标准化协议)	Web 应用 / 后台系统	✓ 验证中
原生 UI 路径	系统 UI / 原生 App	本地 Socket + 签名伴侣	需 macOS TCC 权限	中 (受系统 / API 限制)	原生 App / 系统级任务	📦 规划中

结论

本节验证场景走 Web UI 路径, 通过 Playwright + CDP 实现对小小红书创作后台的完整穿透执行。Peekaboo 原生 UI 路径作为后续扩展能力边界。

第 3 章

锁定业务对象，而不是追逐像素

语义锚点、多候选 Selector 与抗变动定位策略

“

自动化不是记住今天页面长什么样，
而是描述我要找的业务对象有什么特征。

Browser Automation 核心原则

坐标会变，业务对象的语义相对更稳定。

坐标点击 vs 语义定位

坐标点击

示例

`click(1320, 860)`

依赖

分辨率、布局、滚动、弹窗、缩放

问题

很难解释点的是谁

语义定位

文本

找文本"发布"

表单

找 placeholder"填写标题"

控件

找 `role=button` 或 `input[type=file]`

 发布笔记  首页 笔记管理 数据看板  活动中心 笔记灵感 创作学院 创作百科图片编辑 1/18  获取封面建议

OpenClaw 能帮业务团队做什么?

 智能标题 

很多团队不是缺 AI 聊天工具，而是缺一个能把业务流程真正跑起来的数字员工。OpenClaw 的价值在于：它可以长期驻留、记住上下文、按计划执行任务，并通过浏览器自动化接入那些没有 API 的后台系统。

典型场景包括：

1. 运营团队：自动收集竞品动态、整理素材、生成发布草稿。
2. 销售团队：从会议纪要和客户资料里提取线索，自动更新 CRM。
3. 财务团队：读取票据和表格，再登录后台完成填报和截图留档。
4. 管理团队：每天早上自动生成业务简报，把异常项推送给负责人。

它最适合解决的不是单次问答，而是重复、跨系统、有检查闭环的业务流。简单说：把人从机械操作里解放出来，把判断和确认留给人。

#OpenClaw #AI工具 #浏览器自动化 #商业自动化

#开发者选项

#提升客户体验

#个人开发者

#软件开发

#用户体验

更多 # 话题 

@ 用户

 表情

323/1000

笔记预览

封面预览



多候选 selector：给业务对象建立锚点矩阵

/config/selectors.json

```
"title_input_any": [  
  {"kind": "placeholder", "value": "填写标题"},  
  {"kind": "placeholder", "value": "请输入标题"},  
  {"kind": "css", "value": "input[placeholder*='标题']"},  
  {"kind": "css", "value": "textarea[placeholder*='标题']"}  
]
```

锚点矩阵

同一个业务对象，准备多个候选定位方式。前端改版只要其中一个命中就够了。

语义优先

优先使用人能理解的 placeholder 文本，可读性高，维护成本低。

CSS 兜底

再用 CSS 属性特征做后备定位，确保至少有一种方式能命中。

定位方式的稳定性排序

定位方式	例子	稳定性	适合场景
业务文本	<code>get_by_text("上传图文")</code>	中高	菜单、标签、按钮
Placeholder	<code>get_by_placeholder("填写标题")</code>	高	表单输入框
Role + Name	<code>get_by_role("button", name="发布")</code>	高	标准按钮
CSS 特征	<code>input[type='file']</code>	中	上传控件
XPath 层级	<code>div[3]/div[2]/button</code>	低	临时调试
坐标点击	<code>(x, y)</code>	最低	最后兜底

核心观点：越接近业务语义，稳定性越高，越容易维护。

定位工具函数：把"尝试多个候选"变成通用能力

/src/locator_utils.py

```
async def first_visible(page, specs, timeout_ms=1500):
    errors = []
    for spec in specs:
        try:
            locator = build_locator(page, spec)
            await locator.first.wait_for(
                state="visible",
                timeout=timeout_ms
            )
            return locator.first
        except Exception as exc:
            errors.append(f"{spec}: {exc}")
    raise LocatorNotFound("; ".join(errors))
```

逐个尝试

按候选 selector 顺序寻找元素，第一个命中就返回。

可见才返回

只返回真正 visible 的元素，避免定位到隐藏节点。

失败可排查

保留所有候选的错误信息，selector 失效时一眼看出哪个断了。

登录与验证码：不要把边界设计错

现实约束

登录态

会过期，二维码有时效

验证码

可能出现滑块、人机校验

云服务器

IP 可能触发额外验证

正确姿势

不绕过

不绕过验证码，不操作未授权账号

人工接管

截图 → payload → 龙虾发飞书群 → 扫码

双模式

保留 draft / publish 两种执行模式

业务自动化要可控，不是盲目无人化。

第 4 章

一键部署、对话发布、扫码接管

三步跑通小红书自动发布：部署 skill → 飞书对话 → 扫码登录

DjangoPeng / agentic-ai

Q Type to search

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Files

main

Go to file

openclaw-heartbeat

openclaw-im

openclaw-infra

openclaw-models

openclaw-skills

openclaw-soul

xhs-auto-publisher

config

deploy

docs

examples

scripts

src

.gitignore

README.md

SKILL.md

lesson12-lab.md

requirements.txt

.gitignore

LICENSE

README.md

openclaw.json.example

agentic-ai / xhs-auto-publisher

Add file

DjangoPeng and claude refactor(setup): 用软链接替代 cp, 去掉冗余的目录复制

9fc6bbd · 3 days ago

History

Name	Last commit message	Last commit date
..		
config	feat(lesson12): 新增小红书云端自动发布 Agent 目录	4 days ago
deploy	refactor(setup): 用软链接替代 cp, 去掉冗余的目录复制	3 days ago
docs	fix(xhs-auto-publisher): 部署路径改为 \\${HOME}/projects, 不再硬编码 /root	4 days ago
examples	feat(lesson12): 新增小红书云端自动发布 Agent 目录	4 days ago
scripts	feat(lesson12): 新增小红书云端自动发布 Agent 目录	4 days ago
src	feat(lesson12): 新增小红书云端自动发布 Agent 目录	4 days ago
.gitignore	feat(lesson12): 新增小红书云端自动发布 Agent 目录	4 days ago
README.md	fix(xhs-auto-publisher): 部署路径改为 \\${HOME}/projects, 不再硬编码 /root	4 days ago
SKILL.md	fix(xhs-auto-publisher): 部署路径改为 \\${HOME}/projects, 不再硬编码 /root	4 days ago
lesson12-lab.md	feat(lesson12): 新增实验三——自定义内容发布完整链路	4 days ago
requirements.txt	feat(lesson12): 新增小红书云端自动发布 Agent 目录	4 days ago

README.md

xhs-auto-publisher

一个面向云服务器的小红书图文自动发布 Skill。

这套项目的定位很明确：

- 在 Linux 云服务器上运行 Playwright/Chromium

三步跑通小红书自动发布

Step 1: 一键部署 —— 把 setup.sh 的 Prompt 发给龙虾, 它在云服务器上自动拉 GitHub 课程项目仓库、装依赖、初始化环境

Step 2: 对话触发 —— 告诉龙虾你要发布的内容, 它生成 JSON、调用 Skill、自动执行

Step 3: 扫码登录 —— 龙虾把二维码图片发到飞书群, 你扫码后任务继续

核心范式: 口述意图 → 龙虾生成结构化数据 → Skill 自动执行

Step 1: 一键部署 —— 发给龙虾的 Prompt

发给龙虾的完整 Prompt

帮我在云服务器上部署并运行一个 Skill。

第一步，一键部署：

```
curl -fsSL
```

```
https://raw.githubusercontent.com/Django
```

```
Peng/agent-ai/main/xhs-auto-
```

```
publisher/deploy/setup.sh | bash
```

昨天 16:44



帮我在云服务器上部署并运行一个 Skill。

第一步，一键部署：

```
curl -fsSL https://raw.githubusercontent.com/DjangoPeng/agent-ai/main/xhs-auto-publisher/deploy/setup.sh | bash
```

第二步，运行：

```
bash ~/projects/xhs-auto-publisher/deploy/run_with_xvfb.sh
```

每完成一步汇报结果。跑到需要扫码登录时停下来，把二维码图片路径告诉我。

raw.githubusercontent.com

```
#!/usr/bin/env bash set -euo pipefail
REPO_URL="https://github.com/DjangoPeng/agent-a..."
```



先跑部署脚本，我跟一下结果。

第一步执行完了，但部署失败，卡在 Python 依赖安装。脚本前半段已经完成：仓库拉到 /root/projects/agent-ai，系统依赖和 xvfb 也装好了。

失败点是 pip 源解析异常：它在用 <https://mirrors.ivoles.com/pypi/simple/>，域名解析失败，导致 playwright>=1.44 装不上，所以还不能进入第二步运行。

如果你同意，我下一步直接把这个项目的 pip 源切回官方 PyPI 或可用镜像，然后继续安装并启动；跑到扫码登录时我会把二维码图片路径发你。第一步执行完了，但部署失败，卡在 Python 依赖安装。脚本前半段已经完成：仓库拉到 /root/projects/agent-ai，系统依赖和 xvfb 也装好了。

失败点是 pip 源解析异常：它在用 <https://mirrors.ivoles.com/pypi/simple/>，域名解析失败，导致 playwright>=1.44 装不上，所以还不能进入第二步运行。

如果你同意，我下一步直接把这个项目的 pip 源切回官方 PyPI 或可用镜像，然后继续安装并启动；跑到扫码登录时我会把二维码图片路径发你。

Okay, so the async command that completed was the business-news-final.sh script, which generated the 2026-05-22 business news report and copied it to memory/2026-05-22-briefing.md. Let's check if there's anything else we need to relay to the user. Since this is the completion of the async command, we should tell the user that the business news final report is ready.

Okay, the async command has completed. The business news final report for 2026-05-22 is ready and has been copied to memory/2026-05-22-briefing.md. Here's a summary of today's top news:

1. How Glance turns hours of video into mobile-ready clips with AI (Google)
2. Using KagI Search with Low Vision (HN top ~2)
3. Indexing a year of video locally on a 2021 MacBook with Gemma4-31B (HN top ~7)
4. Beyond the engine: 10 open source projects shaping how games actually get made (GitHub)
5. The top announcements for startups from Google I/O '26 (Google)

Let me know if you need more details.



好，你用官方的 pip 源吧



我直接切到官方 PyPI 继续补装依赖，装完就启动。

依赖已经补装成功，正在下载 Playwright 的浏览器内核。下一个结果出来我就继续汇报。

DjangoPeng / agentic-ai

Q

Type to search

<>

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Files

main

+

Q

Q

Go to file

T

>

openclaw-heartbeat

>

openclaw-im

>

openclaw-infra

>

openclaw-models

>

openclaw-skills

>

openclaw-soul

>

xhs-auto-publisher

>

config

>

deploy

>

systemd

bootstrap_project.sh

env.example

install_system_ubuntu.sh

run_with_xvfb.sh

setup.sh

>

docs

>

examples

>

scripts

publish_xhs.py

>

src

.gitignore

README.md

agentic-ai / xhs-auto-publisher / deploy / setup.sh

DjangoPeng and claude refactor(setup): 用软链接替代 cp, 去掉冗余的目录复制

9fc6bbd · 2 hours ago

History

Code

Blame

Executable File · 34 lines (28 loc) · 1.18 KB

Raw

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 REPO_URL="https://github.com/DjangoPeng/agentic-ai"
5 REPO_DIR="${HOME}/projects/agentic-ai"
6 PROJECT_ROOT="${HOME}/projects/xhs-auto-publisher"
7
8 echo "[setup] step 1/3 - clone or update repo"
9 if [ -d "${REPO_DIR}/.git" ]; then
10     git -C "${REPO_DIR}" pull
11 else
12     mkdir -p "${HOME}/projects"
13     git clone "${REPO_URL}" "${REPO_DIR}"
14 fi
15
16 echo "[setup] step 2/3 - link xhs-auto-publisher"
17 if [ -d "${PROJECT_ROOT}" ] && [ ! -L "${PROJECT_ROOT}" ]; then
18     echo "[setup] removing old copy, replacing with symlink"
19     rm -rf "${PROJECT_ROOT}"
20 fi
21 ln -sfn "${REPO_DIR}/xhs-auto-publisher" "${PROJECT_ROOT}"
22
23 echo "[setup] writing .env (MODE=draft)"
24 cp "${PROJECT_ROOT}/deploy/env.example" "${PROJECT_ROOT}/.env"
25 sed -i 's/^MODE=.*$/MODE=draft/' "${PROJECT_ROOT}/.env"
26
27 echo "[setup] step 3/3 - install system dependencies and init project"
28 bash "${PROJECT_ROOT}/deploy/install_system_ubuntu.sh"
29 bash "${PROJECT_ROOT}/deploy/bootstrap_project.sh"
30
31 echo "[setup] done - project ready at ${PROJECT_ROOT}"
32 echo "[setup] code lives in ${REPO_DIR}/xhs-auto-publisher"
33 echo "[setup] to update: git -C ${REPO_DIR} pull"
34 echo "[setup] run with: bash ${PROJECT_ROOT}/deploy/run_with_xvfb.sh"
```

Step 2: 对话触发 —— 发布样例文章

发给龙虾的 Prompt

第二步，运行：

```
bash ~/projects/xhs-auto-publisher/  
    deploy/run_with_xvfb.sh
```

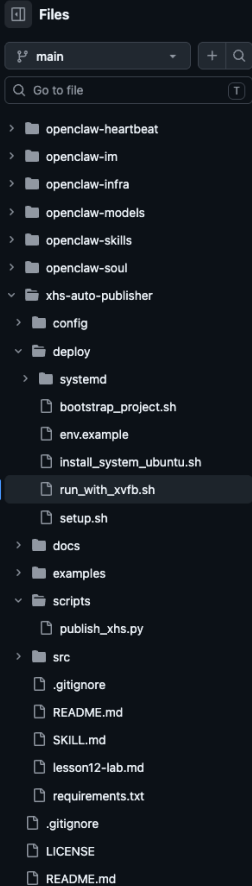
每完成一步回报结果。

跑到需要扫码登录时：

1. 读取 `~/projects/xhs-auto-publisher/runtime/lobster-notify/<run_id>/login_qr.payload.json`
2. 取出 `delivery.path` 字段对应的图片文件
3. 把那张二维码图片直接发到飞书群
(图片消息，不是路径文字)
4. 把 `caption_lines` 的内容作为说明文字发出
5. 等待我确认扫码完成后再继续执行



Step 2 续：对话触发 —— 发布样例文章



agentica-ai / xhs-auto-publisher / deploy / run_with_xvfb.sh

DjangoPeng and claude fix(xhs-auto-publisher): 部署路径改为 \\${HOME}/projects, 不再硬编码 /root

Code Blame 43 lines (36 loc) · 1.09 KB

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 PROJECT_ROOT="${PROJECT_ROOT:-${HOME}/projects/xhs-auto-publisher}"
5 PYTHON_BIN="${PROJECT_ROOT}/.venv/bin/python"
6 CONTENT_PATH="${1:-${PROJECT_ROOT}/examples/openclaw_business_content.json}"
7 MODE="${MODE:-publish}"
8 LOGIN_TIMEOUT="${LOGIN_TIMEOUT:-300}"
9 DISPLAY_NUM="${DISPLAY_NUM:-99}"
10
11 if [ -f "${PROJECT_ROOT}/.env" ]; then
12     # shellcheck disable=SC1091
13     set -a
14     . "${PROJECT_ROOT}/.env"
15     set +a
16 fi
17
18 echo "[run] project root: ${PROJECT_ROOT}"
19 echo "[run] content path: ${CONTENT_PATH}"
20 echo "[run] mode: ${MODE}"
21 echo "[run] login timeout: ${LOGIN_TIMEOUT}"
22
23 if [ ! -x "${PYTHON_BIN}" ]; then
24     echo "[run] python environment not found: ${PYTHON_BIN}" >62
25     exit 1
26 fi
27
28 if [ ! -f "${CONTENT_PATH}" ]; then
29     echo "[run] content file not found: ${CONTENT_PATH}" >62
30     exit 1
31 fi
32
33 cd "${PROJECT_ROOT}"
34
35 exec xvfb-run \
36     --auto-servernum \
37     --server-num="${DISPLAY_NUM}" \
38     --server-args="-screen 0 1440x1080x24" \
39     "${PYTHON_BIN}" \
40     "${PROJECT_ROOT}/scripts/publish_xhs.py" \
41     --content "${CONTENT_PATH}" \
42     --mode "${MODE}" \
43     --login-timeout "${LOGIN_TIMEOUT}"
```

16:16 97

OpenClaw 商业应用场景

没有 API 的后台，也能自动跑流程

OpenClaw 能帮业务团队做什么？

很多团队不是缺 AI 聊天工具，而是缺一个能把业务流程真正跑起来的数字员工。OpenClaw 的价值在于：它可以长期驻留、记住上下文、按计划执行任务，并通过浏览器自动化接入那些没有 API 的后台系统。

Public

Edit & permissions >

Like Save Cmt

16:16 97

很多团队不是缺 AI 聊天工具，而是缺一个能把业务流程真正跑起来的数字员工。OpenClaw 的价值在于：它可以长期驻留、记住上下文、按计划执行任务，并通过浏览器自动化接入那些没有 API 的后台系统。

典型场景包括：

- 运营团队：自动收集竞品动态、整理素材、生成发布草稿。
- 销售团队：从会议纪要和客户资料里提取线索，自动更新 CRM。
- 财务团队：读取票据和表格，再登录后台完成填报和截图留痕。
- 管理团队：每天早上自动生成业务简报，把异常项推送给负责人。

它最适合解决的不是单次问答，而是重复、跨系统、有检查闭环的业务流。简单说：把人从机械操作里解放出来，把判断和确认留给给人。

#OpenClaw #AI工具 #浏览器自动化 #商业自动化

Translate

Yesterday, 17:20 Indonesia 15 views

Comments Likes & Saves

Public

Edit & permissions >

Like Save Cmt

构造登录二维码 payload：把人工接管标准化

login_qr.payload.json

```
{
  "channel": "lobster_channel",
  "kind": "login_qr",
  "platform": "xiaohongshu",
  "run_id": "20260516-153000",
  "delivery": {
    "type": "image_file",
    "path": "runtime/.../login_qr.png",
    "caption_lines": [
      "[XHS] 小红书登录二维码",
      "请扫码完成登录"
    ]
  }
}
```

不直连飞书

项目不直接发 webhook，只产生标准 payload 文件。

标准交接

payload 是给龙虾消费的交接协议，职责解耦。

图片交付

交接对象是本地截图，不需要公网链接。

Step 3: 扫码登录 —— 你唯一需要动手的一步

龙虾发图

二维码图片直接出现在飞书群，不是链接，不需要公网

你扫码

用小红书 App 扫飞书群里的二维码，完成登录授权

Skill 继续

登录成功后，Skill 自动继续执行发布流程

登录态缓存

浏览器 profile 保存登录态，下次运行可能不需要再扫

人工接管是设计出来的流程节点，不是失败。

Skill 内部工程：五道关卡总览

图文入口定位

用业务文本找到上传图文入口，避免停在视频模式

本地图片上传

用 `input[type=file]` 走浏览器原生文件通道，不模拟拖拽

标题与正文定位

多候选 `selector + first_visible`，绑定业务对象而不是 DOM 层级

内容回读验证

填写后读回页面状态，确认编辑器真正接受了内容

提交按钮策略链

四级递进：组件方法 → DOM 打分 → 可见过滤 → 坐标兜底

每一步都用到了 Ch3 的语义锚点和多候选 `selector` 策略

课堂实践的几条红线

先跑 draft，不直接默认真实发布

通过飞书 IM 对话完成，不需要进终端

不把验证码处理讲成绕过技巧

不依赖公网二维码链接

不要求学员拿重要账号做第一次测试

下一章进入动手环节，在飞书群里跑通 draft 链路。

第 5 章

实战： 在飞书跑通小红书全自动发布

手把手带你完成实战闭环

三

DjangoPeng / agentic-ai

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Files

main

Go to file

openclaw-heartbeat

openclaw-im

openclaw-infra

openclaw-models

openclaw-skills

openclaw-soul

xhs-auto-publisher

config

deploy

systemd

bootstrap_project.sh

env.example

install_system_ubuntu.sh

run_with_xvfb.sh

setup.sh

docs

examples

scripts

publish_xhs.py

src

.gitignore

README.md

SKILL.md

lesson12-lab.md

requirements.txt

.gitignore

LICENSE

agent-ai / xhs-auto-publisher / lesson12-lab.md

https://github.com/DjangoPeng/agent-ai/blob/main/xhs-auto-publisher/lesson12-lab.md

DjangoPeng and claude feat(lesson12): 新增实验三——自定义内容发布完整链路

0e209fa · yesterday History

PreviewCodeBlame238 lines (163 loc) · 7.73 KB

Raw

第12节 实验手册：云端部署小红书自动发布 Skill

配套课程：AI 业务流架构师·第12节 云端执行体与跨端登录协作 前置条件：已准备好 DjangoPeng/agent-ai 仓库（xhs-auto-publisher/ 目录），并能让龙虾访问目标云服务器 操作方式：把本实验中的完整 Prompt 复制给龙虾执行，学员不需要直接登录服务器 预计耗时：60-90 分钟

实验目标

1. 理解小红书自动发布 Skill 的云端最小可运行链路：生成二维码截图 -> 生成 payload -> 由龙虾转发到飞书群。

2. 完成一次标准云端部署：拉取仓库、安装系统依赖、初始化项目、复制 .env 、设置 MODE=draft 。

3. 验证 Playwright + Xvfb 能在 Ubuntu 服务器上启动浏览器并跑到扫码登录阶段。

4. 检查二维码截图和 lobster notify payload 是否正确生成。

5. 理解内容 JSON 格式，掌握自定义发布内容的完整链路。

6. 体验核心 Agent 范式：口述意图 -> 龙虾生成结构化数据 -> 自动执行操作。

实验说明

这一节不是要把小红书笔记真正发布出去，而是验证云端 Skill 是否具备最基本的运行能力。

当前项目只走一条链路：

生成二维码截图 -> 生成 payload -> 后续由龙虾把图片发到飞书群

只要成功生成下面两个文件，就说明本节实验完成：

runtime/runs/<run_id>/screenshots/login_qr.pngruntime/lobster-notify/<run_id>/login_qr.payload.json

注意：下面的 Prompt 是本实验的核心执行指令。复制给龙虾时请保持完整，不要拆开改写。

演示：龙虾回报部署结果

仓库已克隆到 ~/projects/xhs-auto-publisher

.venv 创建成功

Playwright Chromium 安装完成

.env 已写入 MODE=draft



在云服务器上执行：

```
curl -fsSL https://raw.githubusercontent.com/DjangoPeng/agent-ai/main/xhs-auto-publisher/deploy/setup.sh | bash
```

每一步完成后汇报结果，如果失败贴出报错。

```
raw.githubusercontent.com
#!/usr/bin/env bash set -euo pipefail
REPO_URL="https://github.com/DjangoPeng/agent-ai"
```

2 条回复



已开始执行安装脚本，我先看首批输出。

前两步已完成：仓库已更新，项目软链接和 .env 已写入。现在在执行第 3 步，正在安装系统依赖。

系统依赖安装完成，没有报错；当前进入项目初始化阶段，正在创建 Python 虚拟环境。

回复 彭靖田：在云服务器上执行： curl -fsSL https://raw.githubusercontent.com/DjangoPeng/agent-ai/main

已开始执行安装脚本，我先看首批输出。前两步已完成：仓库已更新，项目软链接和 .env 已写入。现在在执行第 3 步，正在安装系统依赖。系统依赖安装完成，没有报错；当前进入项目初始化阶段，正在创建 Python 虚拟环境。前面步骤都成功了，但现在在“安装 Python 依赖”这一步出现网络解析问题，脚本还在重试。当前报错是：

```
1 WARNING: Retrying ... Failed to establish a new connection: [Err
```

它在访问的源是：

```
1 https://mirrors.ivolces.com/pypi/simple/
```

我继续等这一步最终结果。执行结果如下：

1. 第 1 步完成：更新仓库成功

```
1 [setup] step 1/3 - clone or update repo
2 Updating 0ed4915..9fc6bbd
3 Fast-forward
4 ...
5 From https://github.com/DjangoPeng/agent-ai
6 0e289fa..9fc6bbd main -> origin/main
```

演示：二维码出现在飞书

Skill 在云端启动浏览器

检测到未登录，截出二维码

生成 login_qr.payload.json

龙虾把图片发到群里



第二步，运行：

```
bash ~/projects/xhs-auto-publisher/deploy/run_with_xvfb.sh
```

每完成一步回报结果。

跑到需要扫码登录时：

1. 读取 ~/projects/xhs-auto-publisher/runtime/lobster-notify/<run_id>/login_qr.payload.json
2. 取出 delivery.path 字段对应的图片文件
3. 把那二维码图片直接发到飞书群（图片消息，不是路径文字）
4. 把 delivery.caption_lines 的内容作为说明文字一并发出
5. 等待我确认扫码完成后再继续执行

龙虾助理

2 条回复



我先启动脚本并盯住输出，跑到登录二维码那一步我会去取图并发给您。

|| 回复 彭靖田：第二步，运行： bash ~/projects/xhs-auto-publisher/deploy/run_with_xvfb.sh 每完成一...

main

我先启动脚本并盯住输出，跑到登录二维码那一步我会去取图并发给您。第 1 步已开始，脚本启动成功，当前参数如下：

```
1 [run] project root: /root/projects/xhs-auto-publisher
2 [run] content path: /root/projects/xhs-auto-publisher/examples/o
3 [run] mode: draft
4 [run] login timeout: 300
```

我继续等下一步输出。运行进程还活着，但暂时没有新日志。我去检查 runtime/lobster-notify 目录里是否已经生成登录二维码 payload。runtime/lobster-notify 目录目前还是空的，说明还没走到生成二维码通知文件那一步。我继续盯运行日志，一出现二维码 payload 就立刻取图并发给您。目前脚本仍在运行，但还没有进入扫码通知阶段，也没有新的标准输出。

演示：二维码出现在飞书

| 回复 彭靖田：第二步，运行： `bash ~/projects/xhs-auto-publisher/deploy/run_with_xvfb.sh` 每完成一...

main

目前脚本仍在运行，但还没有进入扫码通知阶段，也没有新的标准输出。

现状：

- 1 - 运行进程仍存活
- 2 - lobster-notify 目录为空
- 3 - 尚未生成 login_qr.payload.json

我会继续等待；一旦二维码文件出现，就按您的要求把图片和说明文字发出来。

Agent: main | Model: gpt-5.4 | Provider: hotai

还在跑，日志依然没有新增输出。我再直接检查一次运行目录里最近生成的文件，确认是不是二维码写到别的位置了。

二维码通知文件已经生成了。我现在读取 payload，取出图片和说明文字，然后直接把图片发给您。

[XHS Cloud Login] 小红书登录二维码

Run ID: 20260523-161004

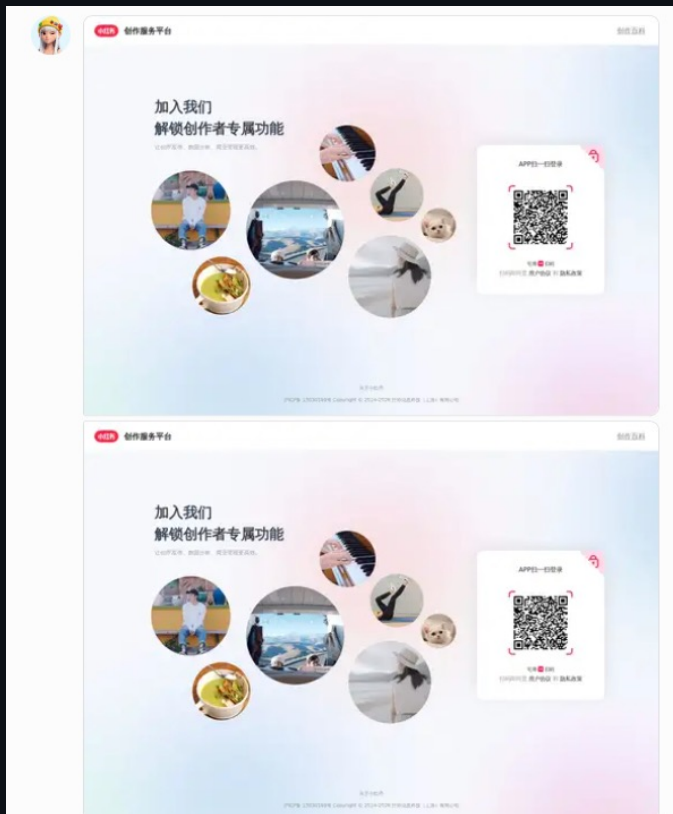
图片路径: /root/projects/agent-ai/xhs-auto-publisher/runtime/runs/20260523-161004/screenshots/login_qr.png

请扫码后回复我“已完成扫码”，我再继续执行。

I don't see any command output in the system messages above. Could you please share the command output you'd like me to relay, or clarify which async command you're referring to?

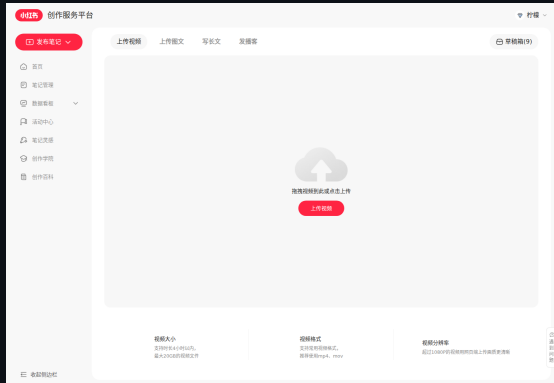


重新生成二维码，并且把图片发送到这里，

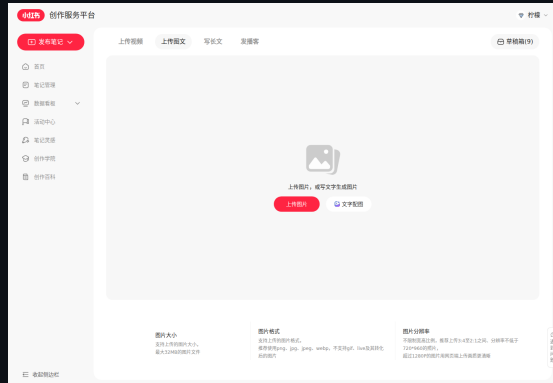


运行结果：Agent 后台动作

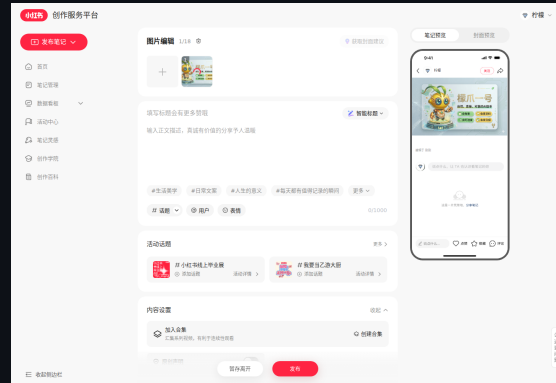
步骤一：打开发布界面



步骤二：切换为上传图片



步骤三：图片上传



运行结果：后台动作已完成

标题已填入

正文已填入

话题已追加

图片已上传



运行产物：Skill 留下了什么

`actions.jsonl`

每一步操作的日志

`result.json`

最终状态：draft_ready / published

`content.normalized.json`

标准化后的内容 JSON

`screenshots/*.png`

关键步骤截图

`dom/*.html`

页面 DOM 快照

让自动化结果可复盘、可审计、可复跑。

Agent 发布的小红书文案使用 JSON 格式

my_content.json

```
{  
  "title": "标题, 最多 80 字",  
  "body": "正文, 最多 2000 字",  
  "topics": ["话题1", "话题2"],  
  "images": ["图片路径"],  
  "mode": "draft"  
}
```

口述即可

告诉龙虾你想发什么，它帮你生成这份 JSON。

先 draft 再 publish

确认内容填写正确后再改 mode 为 publish。

图片三种来源

项目自带样例图 / 公网 URL 下载 / 本机上传。

小红书防封号建议

标题

按 20 字以内做校验；核心关键词前置；避免标题党、极限词和夸张承诺。

正文

普通图文建议 300-800 字；分 3-5 个短段；不要把正文只写成摘要。

话题

建议 2-5 个精准话题；以 # 开头；多个话题用空格或换行分隔。

图片

课堂保守用 1-9 张；首图承担点击理由；推荐竖图 3:4，图片清晰无水印。

合规

避免侵权搬运、站外导流、虚假夸大、伪科学、收益/疗效承诺和敏感信息。

自动化

脚本先校验内容对象，再进入浏览器；失败时保留 JSON、截图、DOM 和日志。

核心原则：课堂按保守值先跑通上传链路；后台上限变化时，以创作者平台现场提示为准。

上手实操的常见卡点

卡点	现象	处理建议
GitHub 拉取失败	setup.sh 报错	检查服务器网络
系统依赖安装失败	apt 报错	确认 root 用户和 Ubuntu 版本
Chromium 初始化失败	缺少系统库	重新执行 setup.sh
没有生成二维码	runtime 目录为空	检查 run_with_xvfb.sh 是否启动
扫码后页面无反应	登录轮询超时	增大 LOGIN_TIMEOUT
页面结构变化	找不到输入框	更新 selectors.json

遇到卡点时让龙虾回报具体错误，对照排查。

云端运行的关键：Xvfb 虚拟屏幕

问题

云服务器没有显示器，有头浏览器启动就报错

方案

Xvfb (X Virtual Frame Buffer) 提供虚拟显示环境

一行核心

xvfb-run 包住 publish_xhs.py，让它以为自己有屏幕

学生不需要手动配置 Xvfb，setup.sh 已经装好了。

deploy/run_with_xvfb.sh 核心

```
exec xvfb-run --auto-servernum \  
  --server-args="-screen 0 1440x1000x24" \  
  python publish_xhs.py --content $CONTENT --mode $MODE
```

云端通知链路：执行器与消息系统解耦

执行器只负责产出结果（payload），消息系统负责投递（delivery）



结论



二维码不需要公网 URL
直接发送图片即可



链路更短
减少中间环节



更安全
避免临时 URL 泄露风险



解耦设计
执行器与消息系统拉演进



执行器（项目）



payload 文件



龙虾（投递系统）



飞书群（用户）

从小红书发布后台迁移到企业核心系统

验证场景模块	企业后台对应物	可复用思想
内容 JSON	报销单 / 客户资料 / 工单	输入结构化
图片上传	发票 / 合同 / 附件	文件路径校验与上传
标题正文填写	表单字段填写	selector 配置化
二维码登录	SSO / 短信 / MFA	人工接管边界
draft 模式	提交前预览	先填充，后确认
runtime 产物	审批留痕	可复盘、可审计

小红书只是验证场景，底层架构可以平移到任何没有 API 的业务后台。

第 6 章

总结与思考

核心收获与课后作业

本节核心收获

① 系统 UI 是无 API 时代的最后接口

当后台没有开放接口，浏览器界面就是业务流可以接入的执行面。

② Browser Automation 是 AI 执行层，不是简单脚本

CDP、DOM、视觉锚点、profile、payload 共同构成可托管能力。

③ 稳定穿透依赖语义锚点，而不是像素记忆

文本、placeholder、role、CSS 特征和多候选 selector 是核心方法。

④ 真实平台必须设计人机接管边界

登录扫码、验证码、最终提交确认，应该被设计成可控流程。

课后作业：小红书自动化发布实战

1

必做

复现课堂实战

在飞书群中通过 OpenClaw 对话完成一次 draft 发布：发送图文内容（标题、正文、话题、配图）→ OpenClaw 调用 xhs-auto-publisher Skill → 扫码登录 → 确认内容已填入后台并停在发布前。

交付：飞书对话截图、扫码截图、before_publish 截图

2

必做

发布一篇自己写的小红书文章

自己撰写一篇原创图文（标题≤20字，正文 300-800 字，含话题和配图），通过 OpenClaw 用 publish 模式完成真实发布。

交付：飞书对话记录、发布后的笔记链接

3

选做

小红书自动发布 Agent 接入 Cron 任务，实现 AI 自媒体运营

将之前课程中完成的资讯日报或 GitHub 趋势日报接入 Cron 任务，每天通过 OpenClaw 自动生成一篇图文并发布到小红书。

交付：crontab / Heartbeat 配置、至少连续 3 天的发布截图、简要运维日志